Shaikh Zeeshan Murshed

Senior Frontend Engineer | React | TypeScript | Full-Stack

Montréal, QC, Canada | Open to Remote | sk.zeeshan1992@gmail.com | linkedin.com/in/skizsk | github.com/sk-izsk | izsk.dev | izsk.hashnode.dev

Languages: English, Bengali, Hindi

Summary

Senior frontend engineer with 7+ years shipping React and TypeScript products for enterprise SaaS, now extending into Node, React Native and AI-assisted engineering.

Technical Skills

Languages: JavaScript (ES2022+), TypeScript, Python, HTML5, CSS3
Frameworks: React, Next.js (App Router, SSR/SSG, server actions, middleware), Node.js, Express, FastAPI, GraphQL, React Native (Expo)
State / Data: TanStack Query (React Query), Redux Toolkit, Zustand
UI Libraries: Material-UI (MUI), Ant Design, Tailwind CSS, Shadcn, NativeWind, Styled-Components, Bootstrap, JSS
Testing: Jest, Vitest, Cypress (E2E), React Testing Library, React Native Testing Library, Enzyme, Mocha, Chai, Nock
Tools / DevOps: Git, GitLab CI/CD, Docker, Webpack, Vite, Storybook, Postman, Figma, Jira, Notion, Bun
Visualization / 3D: Three.js, React Three Fiber, Drei, Mafs, Framer Motion
Real-Time: Socket.IO, WebSockets, Server-Sent Events (SSE)
Analytics / Monitoring: Google Analytics, Mixpanel, Sentry (React Native and web), Core Web Vitals
Databases: PostgreSQL, MongoDB, Drizzle ORM, Upstash Redis
Auth / Cloud: Clerk, JWT, AWS (S3, Lambda), Cloudinary, Heroku, Netlify, Render
AI & Agentic Tooling: Custom MCP servers (Storybook, Figma, GitHub, SDK docs), custom AI skills, AI-assisted code review, prompt engineering, LLM integration (Groq, OpenAI-compatible APIs), RAG and agent workflows

Experience

Applied AI & Full-Stack Engineering Mar 2026 – Present
AI data platforms, product builds, technical writing *Montréal, QC*

- Evaluate and review LLM outputs on AI data platforms, scoring answers for correctness and reasoning quality so model teams get cleaner training and evaluation data.
- Publish a technical post every Monday on advanced React patterns and AI engineering (10+ so far at izsk.hashnode.dev), which keeps my own understanding sharp and gives readers working examples.
- Deepen the full-stack and AI side of my work through hands-on builds in RAG, agent workflows, FastAPI and Node, with the larger ones listed under Projects.

Software Engineer Nov 2020 – Mar 2026
SwiftConnect — Montréal, QC *Enterprise access & security management SaaS platform*

- Replaced copy-pasted API code in every project with one versioned Node.js SDK and a React SDK on top of it, so a backend change became one fix and a version bump. Almost the whole front-end team adopted them, and when the company began selling its API, third-party customers integrated through the same SDKs.
- Built a custom MCP server over the SDK docs so engineers query API structure and usage from their editor instead of going back and forth on documentation.
- Folded duplicated UI code from three projects into one component library I built from scratch (roughly a third less duplicated code). Components are now checked visually and tested in Storybook, with accessibility handled in one place, before any app imports them, and one fix lands everywhere. Teammates started contributing in the same pattern.

- Picked Next.js for a customer-facing meeting-room booking platform because load speed and SEO mattered, then built it from scratch (routing, state management, base patterns). Static pages stay static and the rest render on the server, which gave customers faster loads and better SEO. It uses API routes, server actions and middleware, and ran in production for about 1.5 years.
- Exposed the component docs through Storybook's MCP, so developers pull usage straight into the editor.
- Cut design revision cycles by roughly 25 to 30% by keeping design rules (padding, borders, colours, shading) in a JSON contract and checking each component against Figma, through a custom Figma MCP and skill, before the PR merges to the SDK. The check says whether a mismatch is a design problem or a code one, so designers only get pulled in when the design itself is wrong.
- Brought PR turnaround from about 2–3 days to about 1–1.5 days (roughly 50%) by having a GitHub MCP review flag console logs, lint violations, unused imports and anti-patterns before a human looks. Small PRs often approve straight away, and people still make the final call.
- Wrote custom AI skills carrying each project's patterns, boilerplate and Figma-check rules, so generated code starts in the team's style instead of being rewritten in review. Wrote them for several projects, including a front-end pattern skill and a boilerplate skill.
- Built, from scratch, the internal React Native (Expo) dashboard that lets customer support see failures and respond from company phones, with push notifications when a customer's system fails. I was the only engineer with prior React Native experience, set up navigation and API integration on the existing SDK and design system, built the data-heavy screens, and covered them with React Native Testing Library.
- Gave support, ops and customer teams a React web dashboard whose error table shows what failed in access management, plus Slack alerts into their channels, so they stopped contacting engineers to decode raw logs. I built the UI, the push alerts and the error table as part of the team.
- Cut email template turnaround from 3–4 days to hours or minutes by moving SendGrid templates into shared React components that engineers build and CI/CD deploys, which freed designers from hand-written HTML and manual deploys.
- Replaced a daily forced logout with 15-minute access tokens that renew from a session token on user activity or focus. A session token renews up to 3 times before a new one is issued, and long inactivity still logs the user out. I designed it, built the proof of concept, took team feedback and implemented it with the team, so customers stay signed in longer without weakening security.
- Added event tracking (Google Analytics first, then Mixpanel, depending on the project) so the team could see which features customers used, polish those, and retire unused ones to cut maintenance.
- Set up Sentry on the React Native app first, then on the web apps, covering iOS and Android crashes and hangs, slow loads, JS errors, release tracking, source maps, alert routing and the console logs leading up to each error. Also set up GitLab CI/CD and Core Web Vitals tracking.
- Replaced React Navigation with Expo's file-based router to cut Dependabot alerts and config overhead. I took the risk case to leadership first and did the migration on a branch without changing behaviour.
- Acted as proxy Product Owner for a few sprints. I started a three-way kickoff where design walks through the user flow, then backend and front-end agree an API contract in front of the design and front-end builds on mock data while backend works. Later POs kept the practice and the back and forth between the three teams dropped.
- Mentored 5 or 6 interns and juniors on navigating the project, the team's expectations, long-term product thinking and shipping features on their own; several shipped independently under guidance.
- Built the core access-control web client in React and TypeScript, with Redux Toolkit and TanStack Query handling complex authorization data so the UI stays stable during heavy mutations across multiple product areas.

Full Stack Developer Oct 2020 – Nov 2020
Hatchways — Remote *Agile team of 4*

- Shipped full-stack features in 2-week sprints with a team of four, using React and Material-UI on the front end and Express, Node.js, MongoDB, Cloudinary and JWT authentication on the back end.

Junior Software Developer July 2019 – June 2020
Guiker Inc. — Montréal, QC *Mobile-first apartment rental marketplace startup*

- Cut lease signing from up to a week of agent visits to 1–2 days by building the online lease flow and in-app landlord–tenant chat, which also let landlords and tenants check the lease any time and helped support onboard customers faster.
- Took part in the React and TypeScript rewrite of the main web product (Material-UI, JSS, REST APIs) with Jest test coverage.
- Owned some Node.js/Express routing and middleware logic, which cut front-end-to-backend integration issues.
- Built the 9-page lease form and fixed needless re-renders caused by React Context with memoization.
- Built mobile features for iOS and Android with React Native (Expo), TypeScript and Jest.

PROJECTS

Portfolio Ask AI | *Full-Stack LLM Feature – React + FastAPI* | izsk.dev | github.com/sk-izsk/zsk-ai-resume-api

- Answers recruiter questions straight from my resume: a React 19 / TypeScript / Vite chat frontend and a Python FastAPI backend on Render calling Groq's llama-3.3-70b-versatile.
- Keeps answers grounded with a prompt pipeline that returns JSON only (`answer`, `sources`, `blocked`) and blocks off-topic questions.
- Keeps inference cheap with Upstash Redis caching on versioned keys, IP rate limiting (3 requests per IP per hour) with 429 handling in the UI, and a `compact_profile()` step that sends reduced context instead of the full 54KB JSON.
- Handles Render free-tier cold starts with a `/health` pre-flight call and a loading overlay before the first `POST /chat`.

Whisper Chat | *Full-Stack Real-Time Messaging App* | Live Demo | github.com/sk-izsk/react-native-fullstack-whisper-chat

- Real-time chat on web (React 19 / Vite) and mobile (React Native / Expo) over one shared Express 5 / Socket.IO / MongoDB TypeScript backend, with Clerk auth on all three.
- Kept cache invalidation separate from socket handling by splitting state: TanStack Query for server state, Zustand for socket connection, unread markers, typing indicators and presence.
- Optimistic message sending, Sentry on mobile, socket logic split into modules, and the same behaviour on web and native without duplicating business logic.

Z-Graph | *Interactive Math Visualization Platform* | z-graph.vercel.app | github.com/sk-izsk/z-graph

- Multi-mode graphing app in React 18 / TypeScript covering 17 categories of math (algebra, calculus, polar, parametric, vector fields, Fourier series, Julia and Mandelbrot fractals, 3D implicit surfaces) with live formula editing, sliders and animation.
- Mafs for 2D and Three.js / React Three Fiber for 3D on a shared Zustand layer, with mathjs parsing formulas and Web Workers keeping fractal sampling and surface meshing off the main thread.

ZTable Web | *Interactive Chemistry App* | zsk-periodic.vercel.app | github.com/sk-izsk/zsk-periodic

- Equation balancing runs in a Web Worker, routes lazy-load through TanStack Router, and the 118 periodic table cells use memoized Zustand selectors, so interactions don't trigger broad re-renders.
- Built with React 19, TypeScript, Vite and Bun across 8 features, including 3D atoms (Three.js / React Three Fiber), molar mass parsing, solubility lookup, PDF worksheet export and i18n.

Productify | *Full-Stack Product Management App* | izsk.dev

- React 19 front end (Vite, React Router 7, compound components) on a typed Express 5 REST API with Drizzle ORM and PostgreSQL (Neon), TanStack Query for cache-aware mutations, and Clerk for authentication and role-based authorization.

EDUCATION

Concordia University Montréal, QC
M.Eng., Electrical & Computer Engineering *Sept 2016 – Dec 2018*

- Coursework: Computer Architecture Design, Microprocessor Applications, Digital Signal Processing, Model-Driven Software Engineering, Formal Hardware Verification, Real-Time Applications, Robotics.

DecodeMTL Montréal, QC
Full-Stack Web Development Bootcamp *April 2019 – May 2019*

- Full-stack web program covering JavaScript, React, Node.js, Express, MongoDB, HTML5 and CSS3.